

Tarea 7: Validador y Extractor de Datos

Semana 7: Expresiones Regulares (RegEx)

NLPY-1: Fundamentos de Python

1. Introducción

Las **expresiones regulares** (regular expressions o regex) son una de las herramientas más poderosas en la programación moderna. Permiten buscar, validar y extraer patrones de texto de manera eficiente y elegante. Son ampliamente utilizadas en validación de formularios, procesamiento de logs, web scraping, análisis de datos y mucho más.

En esta semana aprenderás a dominar las expresiones regulares en Python utilizando el módulo `re`. Desarrollarás un sistema completo de validación y extracción de datos que podrá procesar información de diferentes formatos y fuentes.

1.1. ¿Por qué son importantes las Expresiones Regulares?

- **Validación de datos:** Verificar que emails, teléfonos, cédulas, etc. tengan el formato correcto
- **Extracción de información:** Obtener datos específicos de textos grandes o documentos
- **Búsqueda avanzada:** Encontrar patrones complejos que serían difíciles con métodos tradicionales
- **Transformación de texto:** Reemplazar o reformatear información de manera masiva
- **Parsing de logs:** Analizar archivos de registro de sistemas y aplicaciones

1.2. Objetivos de Aprendizaje

Al completar esta tarea, el estudiante será capaz de:

- Comprender la sintaxis y estructura de las expresiones regulares
- Utilizar metacaracteres, cuantificadores y clases de caracteres
- Aplicar las funciones del módulo `re`: `match()`, `search()`, `findall()`, `sub()`
- Crear patrones para validar formatos costarricenses (cédulas, teléfonos, placas)
- Usar grupos de captura y grupos con nombre
- Implementar flags para modificar el comportamiento de búsqueda
- Extraer información estructurada de texto no estructurado
- Aplicar expresiones regulares para limpieza y sanitización de datos

2. Descripción del Proyecto

Desarrollarás un **Sistema de Validación y Extracción de Datos** que procese diferentes tipos de información usando expresiones regulares. El sistema debe validar formatos costarricenses, extraer datos de textos, y generar reportes estructurados.

2.1. Funcionalidades Requeridas

2.1.1. 1. Validadores (20 puntos - Funcionalidad)

Implementar validadores para formatos costarricenses usando regex:

A) Validador de Cédulas (5 puntos)

- Cédula física: 1-2345-6789 (formato: D-DDDD-DDDD)
- Cédula jurídica: 3-101-123456 (formato: D-DDD-DDDDDD)
- DIMEX: 123456789012 (12 dígitos)
- NITE: 1234567890 (10 dígitos)
- Validar formato y estructura correcta

B) Validador de Teléfonos (4 puntos)

- Formato nacional: 8888-8888, 2222-2222
- Con código de país: +506 8888-8888, +506-8888-8888
- Sin guiones: 88888888
- Validar que inicie con 2, 4, 5, 6, 7 u 8

C) Validador de Emails (3 puntos)

- Formato estándar: usuario@dominio.com
- Permitir puntos, guiones y guiones bajos en usuario
- Validar dominio y extensión
- Soportar subdominios: usuario@mail.empresa.co.cr

D) Validador de Placas de Vehículos (4 puntos)

- Placas particulares: ABC-123, ABC-1234
- Placas comerciales: C-12345
- Placas públicas: PA-1234, TP-12345
- Placas de motos: A-12345
- Case-insensitive

E) Validador de URLs (4 puntos)

- HTTP/HTTPS: https://www.ejemplo.com
- Con rutas: https://ejemplo.com/ruta/archivo.html
- Con parámetros: https://ejemplo.com?param=valor&otro=123
- Con puertos: https://ejemplo.com:8080

2.1.2. 2. Extractores (20 puntos - Funcionalidad)

Implementar extractores que obtengan información de textos:

A) Extractor de Información Personal (8 puntos)

- Extraer todos los emails de un texto
- Extraer todos los teléfonos
- Extraer todas las cédulas
- Extraer fechas (formatos: DD/MM/YYYY, DD-MM-YYYY, YYYY-MM-DD)
- Retornar listas con los datos encontrados

B) Extractor de Precios y Montos (4 puntos)

- Identificar montos en colones: ₡12,345.50, ₡1.500
- Identificar montos en dólares: \$1,234.56
- Extraer números con separadores de miles
- Capturar valor numérico y moneda

C) Procesador de Logs (8 puntos)

- Procesar líneas de log de formato estándar
- Extraer: timestamp, nivel (INFO/ERROR/WARNING), mensaje
- Ejemplo: [2024-10-24 14:30:15] ERROR: Connection failed
- Generar diccionario estructurado por cada entrada
- Contar ocurrencias por nivel de severidad

2.1.3. 3. Requisitos Técnicos

Uso del Módulo re:

- Usar `re.compile()` para patrones reutilizables
- Aplicar `re.match()` para validación de inicio
- Usar `re.search()` para encontrar patrones en texto
- Aplicar `re.findall()` para extraer múltiples coincidencias
- Usar `re.sub()` para reemplazos y limpieza

Características Avanzadas:

- Implementar grupos de captura con paréntesis
- Usar grupos con nombre: `(?P<nombre>...)`
- Aplicar flags: `re.IGNORECASE`, `re.MULTILINE`, `re.DOTALL`
- Implementar lookahead y lookbehind (opcional)

Organización del Código:

- Función validadora por cada tipo de dato
- Función extractora que retorne listas o diccionarios
- Diccionario global con patrones regex compilados
- Menú interactivo para probar validadores
- Función de reporte que muestre estadísticas

3. Conceptos de Expresiones Regulares

3.1. Metacaracteres Básicos

- `.` - Cualquier carácter (excepto salto de línea)
- `^` - Inicio de cadena o línea
- `$` - Fin de cadena o línea
- `*` - 0 o más repeticiones
- `+` - 1 o más repeticiones
- `?` - 0 o 1 repetición (opcional)
- `{n}` - Exactamente n repeticiones
- `{n,m}` - Entre n y m repeticiones
- `|` - OR lógico
- `[]` - Clase de caracteres
- `()` - Grupo de captura

3.2. Clases de Caracteres

- `[abc]` - a, b, o c
- `[a-z]` - Cualquier letra minúscula
- `[A-Z]` - Cualquier letra mayúscula
- `[0-9]` - Cualquier dígito
- `[^abc]` - Cualquier cosa EXCEPTO a, b, c
- `\d` - Dígito [0-9]
- `\D` - No dígito
- `\w` - Alfanumérico [a-zA-Z0-9_]
- `\W` - No alfanumérico
- `\s` - Espacio en blanco
- `\S` - No espacio

3.3. Ejemplos de Patrones

```

1  # Cedula fisica costarricense: 1-2345-6789
2  patron_cedula = r'^\d-\d{4}-\d{4}$'
3
4  # Telefono: 8888-8888 o 88888888
5  patron_telefono = r'^[2-8]\d{3}-?\d{4}$'
6
7  # Email basico
8  patron_email = r'^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
9
10 # Placa particular: ABC-123 o ABC-1234
11 patron_placa = r'^[A-Z]{3}-\d{3,4}$'
12
13 # Fecha DD/MM/YYYY
14 patron_fecha = r'\d{2}/\d{2}/\d{4}'
15
16 # Monto en colones: 12 ,345.50
17 patron_colones = r'[\d,]+\.\?\d{0,2}'
18
19 # Log entry
20 patron_log = r'\[(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] (\w+): (.+)'

```

Listing 1: Ejemplos de patrones regex comunes

4. Plantilla de Código

4.1. Estructura Base del Sistema

```

1  import re
2  from typing import List, Dict, Optional
3
4  # =====
5  # PATRONES REGEX COMPILADOS
6  # =====
7
8  PATRONES = {
9      # Cedulas
10     'cedula_fisica': re.compile(r''), # TODO: patron para 1-2345-6789
11     'cedula_juridica': re.compile(r''), # TODO: patron para 3-101-123456
12     'dimex': re.compile(r''), # TODO: 12 digitos
13     'nite': re.compile(r''), # TODO: 10 digitos
14
15     # Telefonos
16     'telefono': re.compile(r''), # TODO: patrones variados de telefono
17
18     # Email
19     'email': re.compile(r''), # TODO: patron de email
20
21     # Placas
22     'placa_particular': re.compile(r'', re.IGNORECASE), # TODO: ABC-123
23     'placa_comercial': re.compile(r'', re.IGNORECASE), # TODO: C-12345
24     'placa_moto': re.compile(r'', re.IGNORECASE), # TODO: A-12345
25
26     # URLs

```

```

27     'url': re.compile(r'''), # TODO: http(s)://...
28
29     # Fechas
30     'fecha_slash': re.compile(r'''), # TODO: DD/MM/YYYY
31     'fecha_guion': re.compile(r'''), # TODO: DD-MM-YYYY
32     'fecha_iso': re.compile(r'''), # TODO: YYYY-MM-DD
33
34     # Montos
35     'colones': re.compile(r'''), # TODO: 12,345.50
36     'dolares': re.compile(r'''), # TODO: $1,234.56
37
38     # Logs
39     'log_entry': re.compile(r'') # TODO: [timestamp] NIVEL: mensaje
40 }
41
42 # =====
43 # VALIDADORES
44 # =====
45
46 def validar_cedula(cedula: str) -> Dict[str, any]:
47     """
48     Valida una cedula costarricense (fisica, juridica, DIMEX, NITE).
49
50     Returns:
51         dict: {'valida': bool, 'tipo': str, 'mensaje': str}
52     """
53     # TODO: Probar contra cada patron de cedula
54     # TODO: Retornar tipo identificado
55     pass
56
57 def validar_telefono(telefono: str) -> Dict[str, any]:
58     """
59     Valida un numero de telefono costarricense.
60
61     Returns:
62         dict: {'valido': bool, 'formato': str, 'mensaje': str}
63     """
64     # TODO: Validar patron de telefono
65     # TODO: Verificar que inicie con digito valido
66     pass
67
68 def validar_email(email: str) -> Dict[str, any]:
69     """
70     Valida un email.
71
72     Returns:
73         dict: {'valido': bool, 'usuario': str, 'dominio': str}
74     """
75     # TODO: Validar patron de email
76     # TODO: Extraer usuario y dominio usando grupos
77     pass
78
79 def validar_placa(placa: str) -> Dict[str, any]:
80     """
81     Valida una placa vehicular costarricense.
82

```

```

83     Returns:
84         dict: {'valida': bool, 'tipo': str, 'mensaje': str}
85     """
86     # TODO: Validar contra patrones de placas
87     # TODO: Identificar tipo de placa
88     pass
89
90 def validar_url(url: str) -> Dict[str, any]:
91     """
92     Valida una URL.
93
94     Returns:
95         dict: {'valida': bool, 'protocolo': str, 'dominio': str}
96     """
97     # TODO: Validar patron de URL
98     # TODO: Extraer componentes con grupos
99     pass
100
101 # =====
102 # EXTRACTORES
103 # =====
104
105 def extraer_emails(texto: str) -> List[str]:
106     """Extrae todos los emails de un texto."""
107     # TODO: Usar findall para encontrar todos los emails
108     pass
109
110 def extraer_telefonos(texto: str) -> List[str]:
111     """Extrae todos los telefonos de un texto."""
112     # TODO: Usar findall para encontrar todos los telefonos
113     pass
114
115 def extraer_cedulas(texto: str) -> List[Dict[str, str]]:
116     """Extrae todas las cedulas de un texto."""
117     # TODO: Buscar todos los tipos de cedula
118     # TODO: Retornar lista con tipo y valor
119     pass
120
121 def extraer_fechas(texto: str) -> List[str]:
122     """Extrae todas las fechas de un texto."""
123     # TODO: Buscar todos los formatos de fecha
124     pass
125
126 def extraer_montos(texto: str) -> List[Dict[str, any]]:
127     """
128     Extrae montos con moneda de un texto.
129
130     Returns:
131         list: [{'monto': str, 'moneda': str, 'valor': float}, ...]
132     """
133     # TODO: Extraer montos en colones y dolares
134     # TODO: Convertir a valores numericos
135     pass
136
137 def extraer_datos_personales(texto: str) -> Dict[str, List]:
138     """

```

```

139     Extrae toda la informacion personal de un texto.
140
141     Returns:
142         dict: {
143             'emails': [...],
144             'telefonos': [...],
145             'cedulas': [...],
146             'fechas': [...]
147         }
148     """
149     # TODO: Usar todas las funciones extractoras
150     # TODO: Retornar diccionario organizado
151     pass
152
153 # =====
154 # PROCESADOR DE LOGS
155 # =====
156
157 def procesar_log(texto_log: str) -> Dict[str, any]:
158     """
159     Procesa un archivo de log y extrae informacion estructurada.
160
161     Returns:
162         dict: {
163             'entradas': [...],
164             'estadisticas': {'INFO': n, 'ERROR': n, ...}
165         }
166     """
167     # TODO: Extraer cada linea de log
168     # TODO: Parsear timestamp, nivel, mensaje
169     # TODO: Contar por nivel de severidad
170     pass
171
172 # =====
173 # UTILIDADES
174 # =====
175
176 def limpiar_telefono(telefono: str) -> str:
177     """Limpia un telefono dejando solo digitos."""
178     # TODO: Usar re.sub para eliminar no-digitos
179     pass
180
181 def formatear_cedula(cedula: str) -> str:
182     """Formatea una cedula al formato estandar."""
183     # TODO: Extraer digitos y reformatear
184     pass
185
186 def enmascarar_email(email: str) -> str:
187     """Enmascara un email: usuario@dominio.com -> u****o@dominio.com"""
188     # TODO: Usar grupos para capturar y reemplazar
189     pass
190
191 # =====
192 # MENU INTERACTIVO
193 # =====
194

```

```
195 def menu_validadores():
196     """Menu para probar los validadores."""
197     while True:
198         print("\n" + "="*50)
199         print("VALIDADORES")
200         print("="*50)
201         print("1. Validar cedula")
202         print("2. Validar telefono")
203         print("3. Validar email")
204         print("4. Validar placa")
205         print("5. Validar URL")
206         print("6. Volver")
207
208         # TODO: Implementar logica del menu
209         pass
210
211 def menu_extractores():
212     """Menu para probar los extractores."""
213     # TODO: Implementar menu de extractores
214     pass
215
216 def menu_principal():
217     """Menu principal del sistema."""
218     while True:
219         print("\n" + "="*60)
220         print("SISTEMA DE VALIDACION Y EXTRACCION DE DATOS")
221         print("="*60)
222         print("1. Validadores")
223         print("2. Extractores de texto")
224         print("3. Procesador de logs")
225         print("4. Utilidades")
226         print("5. Ejecutar pruebas")
227         print("6. Salir")
228
229         # TODO: Implementar navegacion del menu
230         pass
231
232 # =====
233 # PRUEBAS
234 # =====
235
236 def ejecutar_pruebas():
237     """Ejecuta casos de prueba para validar el sistema."""
238     print("\n" + "="*60)
239     print("EJECUTANDO PRUEBAS...")
240     print("="*60)
241
242     # Pruebas de cedula
243     cedula_prueba = [
244         "1-2345-6789",      # Fisica valida
245         "3-101-123456",    # Juridica valida
246         "123456789012",    # DIMEX valido
247         "1-234-5678",      # Fisica invalida
248     ]
249
250     # TODO: Ejecutar validaciones y mostrar resultados
```

```

251     # TODO: Probar extractores con textos de ejemplo
252     # TODO: Mostrar estadísticas de éxito/fallo
253     pass
254
255     # =====
256     # MAIN
257     # =====
258
259 if __name__ == "__main__":
260     print("Sistema de Validacion y Extraccion de Datos")
261     print("NLPY-1 - Semana 7: Expresiones Regulares")
262     menu_principal()

```

Listing 2: Plantilla del validador y extractor

4.2. Ejemplo: Implementación de un Validador

Como referencia, aquí hay un ejemplo de cómo implementar el validador de email:

```

1 def validar_email(email: str) -> Dict[str, any]:
2     """
3     Valida un email y extrae sus componentes.
4
5     Args:
6         email: String con el email a validar
7
8     Returns:
9         dict: {
10             'valido': bool,
11             'usuario': str,
12             'dominio': str,
13             'mensaje': str
14         }
15     """
16     # Patron con grupos de captura
17     patron = re.compile(
18         r'^(?P<usuario>[a-zA-Z0-9._-]+)@(?P<dominio>[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})$'
19     )
20
21     match = patron.match(email.strip())
22
23     if match:
24         return {
25             'valido': True,
26             'usuario': match.group('usuario'),
27             'dominio': match.group('dominio'),
28             'mensaje': 'Email valido'
29         }
30     else:
31         return {
32             'valido': False,
33             'usuario': None,
34             'dominio': None,
35             'mensaje': 'Formato de email invalido'
36         }
37

```

```
38 # Ejemplos de uso:
39 print(validar_email("juan.perez@tec.ac.cr"))
40 # {'valido': True, 'usuario': 'juan.perez', 'dominio': 'tec.ac.cr', ...}
41
42 print(validar_email("correo_invalido@"))
43 # {'valido': False, 'usuario': None, 'dominio': None, ...}
```

Listing 3: Ejemplo: Validador de email completo

5. Ejemplos de Ejecución

5.1. Validación de Cédula

```
1 >>> validar_cedula("1-2345-6789")
2 {
3     'valida': True,
4     'tipo': 'Fisica',
5     'mensaje': 'Cedula fisica valida'
6 }
7
8 >>> validar_cedula("3-101-654321")
9 {
10     'valida': True,
11     'tipo': 'Juridica',
12     'mensaje': 'Cedula juridica valida'
13 }
14
15 >>> validar_cedula("1-234-567")
16 {
17     'valida': False,
18     'tipo': None,
19     'mensaje': 'Formato de cedula invalido'
20 }
```

Listing 4: Ejemplo de validación de cédulas

5.2. Extracción de Datos

```
1 texto = """
2 Contactar a Juan Perez al telefono 8888-8888 o al email
3 juan.perez@tec.ac.cr. Cedula: 1-2345-6789.
4 Tambien puede llamar a Maria al 2222-2222 o escribir a
5 maria.rodriguez@email.com. Fecha de nacimiento: 15/03/1990.
6 """
7
8 >>> extraer_datos_personales(texto)
9 {
10     'emails': ['juan.perez@tec.ac.cr', 'maria.rodriguez@email.com'],
11     'telefonos': ['8888-8888', '2222-2222'],
12     'cedulas': [{'tipo': 'fisica', 'valor': '1-2345-6789'}],
13     'fechas': ['15/03/1990']
14 }
```

Listing 5: Ejemplo de extracción de datos personales

5.3. Procesamiento de Logs

```
1 log_texto = """
2 [2024-10-24 14:30:15] INFO: Sistema iniciado correctamente
3 [2024-10-24 14:30:16] INFO: Conectando a base de datos
4 [2024-10-24 14:30:20] ERROR: Conexion fallida, reintentando
5 [2024-10-24 14:30:25] WARNING: Tiempo de respuesta elevado
6 [2024-10-24 14:30:30] INFO: Conexion establecida
7 """
8
9 >>> procesar_log(log_texto)
10 {
11     'entradas': [
12         {
13             'timestamp': '2024-10-24 14:30:15',
14             'nivel': 'INFO',
15             'mensaje': 'Sistema iniciado correctamente'
16         },
17         ...
18     ],
19     'estadisticas': {
20         'INFO': 3,
21         'ERROR': 1,
22         'WARNING': 1
23     },
24     'total': 5
25 }
```

Listing 6: Ejemplo de procesamiento de logs

6. Defensa del Código (60 % de la Nota)

6.1. Aspectos Técnicos a Dominar

1. Sintaxis de Expresiones Regulares (15 puntos)

- Explicar metacaracteres: ., *, +, ?, ^, \$
- ¿Cuál es la diferencia entre * y +? ¿Y entre ? y {0,1}?
- ¿Qué hacen los cuantificadores {n}, {n,}, {n,m}?
- Explicar clases de caracteres: [abc], [a-z], [^abc]
- Diferencia entre \d y [0-9]

2. Módulo re de Python (15 puntos)

- Diferencia entre re.match() y re.search()
- ¿Cuándo usar re.findall() vs re.finditer()?
- ¿Para qué sirve re.compile()? ¿Ventajas de compilar patrones?
- Explicar el uso de re.sub() para reemplazos
- ¿Qué retorna cada función? (objeto Match, lista, string, etc.)

3. Grupos de Captura (10 puntos)

- ¿Qué es un grupo de captura? ¿Cómo se define?
- Diferencia entre grupos normales y grupos con nombre
- ¿Cómo acceder a grupos capturados? (`.group()`, `.groups()`)
- ¿Qué son los grupos no capturadores (`?:...`)?
- Explicar backreferences

4. Flags y Modificadores (10 puntos)

- ¿Para qué sirve `re.IGNORECASE`?
- Explicar `re.MULTILINE` y su efecto en `^` y `$`
- ¿Qué hace `re.DOTALL`?
- ¿Cómo combinar múltiples flags?
- ¿Qué es `re.VERBOSE` y cuándo usarlo?

5. Aplicaciones Prácticas (10 puntos)

- ¿Por qué validar con regex en lugar de métodos string?
- Limitaciones de las expresiones regulares
- Casos donde NO se deben usar regex
- Rendimiento: ¿cuándo compilar patrones?
- Mejores prácticas y patrones comunes

6.2. Preguntas de Defensa Típicas

- `.Explica línea por línea tu patrón regex para validar cédulas físicas`
- `”¿Por qué usaste re.match() en lugar de re.search()?”`
- `”Muestra cómo extraerías el código de área de un teléfono usando grupos”`
- `”¿Qué pasa si no usas r'' (raw string) en tus patrones?”`
- `”Modifica tu patrón de email para que también acepte signos de suma en el usuario”`
- `”¿Cómo harías para que tu validador de placas sea case-insensitive?”`
- `.Explica cómo funciona tu extractor de logs paso a paso`

7. Rúbrica de Evaluación

7.1. Funcionalidad del Código (40 puntos)

Validadores (20 puntos):

- 5 puntos: Validador de cédulas (todos los tipos)
- 4 puntos: Validador de teléfonos (formatos variados)
- 3 puntos: Validador de emails
- 4 puntos: Validador de placas (todos los tipos)
- 4 puntos: Validador de URLs

Extractores (20 puntos):

- 8 puntos: Extractor de datos personales completo
- 4 puntos: Extractor de montos y moneda
- 8 puntos: Procesador de logs con estadísticas

7.2. Defensa del Código (60 puntos)

Sintaxis de Regex (15 puntos):

- 6 puntos: Explica metacaracteres y cuantificadores
- 5 puntos: Comprende clases de caracteres
- 4 puntos: Entiende anclas y límites de palabra

Módulo re (15 puntos):

- 5 puntos: Diferencia entre funciones principales
- 5 puntos: Uso correcto de re.compile()
- 5 puntos: Comprende objetos Match y sus métodos

Grupos y Captura (10 puntos):

- 5 puntos: Implementa grupos de captura correctamente
- 3 puntos: Usa grupos con nombre apropiadamente
- 2 puntos: Accede a grupos capturados

Flags y Técnicas Avanzadas (10 puntos):

- 4 puntos: Aplica flags correctamente
- 3 puntos: Usa raw strings
- 3 puntos: Implementa técnicas avanzadas

Calidad y Organización (10 puntos):

- 3 puntos: Patrones bien documentados
- 3 puntos: Código organizado y limpio
- 2 puntos: Manejo de errores
- 2 puntos: Pruebas y ejemplos

7.3. Penalizaciones

- **-5 puntos:** No usar raw strings (`r''`) en patrones regex
- **-5 puntos:** No usar `re.compile()` para patrones reutilizables
- **-3 puntos:** Validadores que no manejan casos límite
- **-3 puntos:** No usar grupos de captura apropiadamente
- **-2 puntos:** Patrones regex sin comentarios explicativos
- **-2 puntos:** No validar entrada antes de procesar
- **-5 puntos:** Código copiado sin comprensión

8. Desafíos Opcionales (Puntos Extra)

8.1. Nivel Intermedio (+10 puntos cada uno)

1. Validador de Contraseñas Seguras

- Mínimo 8 caracteres
- Al menos una mayúscula, minúscula, número y símbolo
- Usar lookahead assertions: `(?=.*[A-Z])`

2. Extractor de Hashtags y Menciones

- Extraer hashtags de estilo Twitter/Instagram
- Extraer menciones (`@usuario`)
- Validar que no empiecen con números

3. Limpiador de Texto HTML

- Remover todas las etiquetas HTML
- Preservar el contenido entre etiquetas
- Decodificar entidades HTML básicas

8.2. Nivel Avanzado (+15 puntos cada uno)

4. Extractor de Información de Facturas

- Procesar texto de facturas costarricenses
- Extraer: número de factura, fecha, RUC, total
- Generar JSON estructurado

5. Validador de Código de Programación

- Detectar y validar sintaxis Python básica
- Extraer nombres de funciones y variables

- Identificar imports

6. Analizador de Sentimientos Básico

- Identificar palabras positivas/negativas usando regex
- Detectar intensificadores (muy, super, etc.)
- Generar score de sentimiento

8.3. Nivel Experto (+20 puntos)

7. Parser de Lenguaje Natural Simple

- Extraer sujeto, verbo, objeto de oraciones simples
- Usar grupos nombrados extensivamente
- Manejar múltiples estructuras gramaticales

8. Validador y Formateador de Direcciones CR

- Parsear direcciones costarricenses con regex
- Extraer provincia, cantón, distrito, señas
- Normalizar y formatear según estándar

9. Recursos y Referencias

9.1. Documentación

- Módulo re de Python: <https://docs.python.org/3/library/re.html>
- Tutorial de regex: <https://docs.python.org/3/howto/regex.html>
- Regex101 (probador online): <https://regex101.com/>
- RegExr (herramienta visual): <https://regexr.com/>

9.2. Consejos para el Desarrollo

1. **Prueba tus regex:** Usa herramientas online antes de implementar
2. **Empieza simple:** Construye patrones incrementalmente
3. **Usa raw strings:** Siempre prefiere `r''` para patrones
4. **Documenta patrones:** Los regex pueden ser crípticos
5. **Casos de prueba:** Ten ejemplos válidos e inválidos
6. **Compila patrones:** Si los usas múltiples veces

10. Criterios de Entrega

10.1. Archivos Requeridos

1. **validador_extractor.py**: Código principal
2. **pruebas.txt**: Archivo con casos de prueba
3. **logs_ejemplo.txt**: Archivo de log para procesar
4. **README.txt**: Documentación del proyecto

10.2. Contenido del README

- Nombre del estudiante
- Descripción de patrones regex implementados
- Instrucciones de uso
- Ejemplos de validación
- Desafíos opcionales completados
- Limitaciones conocidas

*”Las expresiones regulares son como el kung-fu:
parecen mágicas hasta que comprendes los fundamentos.”*

— Adaptación de Jeff Atwood

¡Éxito con tus expresiones regulares!

Recuerda: Un buen patrón regex puede ahorrarte miles de líneas de código.